
用户信息管理平台

安全漏洞挖掘与修复报告

<http://10.133.25.191:5000>

项目版本：V6.0 — 权限提升 + 业务逻辑漏洞专题

报告日期：2026年7月10日

今日课程：Day4:权限提升与业务逻辑漏洞深度解析

技术栈：Python Flask / IDOR / 水平越权 / 参数篡改 / SQL注入

作者：Bear

一、课程背景与知识概述

权限提升漏洞和业务逻辑漏洞是OWASP Top 10 2021中排名第一的"访问控制失效" (A01:2021-Broken Access Control) 的核心内容。与SQL注入、XSS等传统注入类漏洞不同，这两类漏洞更侧重于应用程序的"权限模型设计缺陷"和"业务流程校验缺陷"，攻击者不需要使用复杂的编码技巧，往往只需要修改URL中的一个数字或表单中的一个字段值，就能获得不正当的访问权限或经济利益。

1.1 权限提升漏洞 (IDOR/越权)

定义：权限提升 (Privilege Escalation) 指攻击者通过某种手段，获取到本不属于自己的更高权限或他人数据。在Web应用中，最常见的表现形式是IDOR (Insecure Direct Object Reference，不安全的直接对象引用)。

IDOR原理：当应用程序直接使用用户提供的输入 (如URL参数、表单字段) 来访问数据库中的对象 (如用户资料、订单记录)，且没有验证当前用户是否有权访问该对象时，就产生了IDOR漏洞。攻击者只需修改参数值 (如将 user_id=2 改为 user_id=1)，就能访问其他用户的数据。

分类：

- (1) 水平越权：攻击者访问同级别其他用户的数据 (如普通用户A查看普通用户B的资料)
- (2) 垂直越权：攻击者访问更高级别用户的功能或数据 (如普通用户执行管理员操作)

1.2 业务逻辑漏洞

定义：业务逻辑漏洞是指应用程序在处理业务流程时，由于对业务规则的校验不严谨，导致攻击者可以通过非预期的操作方式获取利益。

常见类型：

- (1) 金额/数量可为负：充值、转账、下单等涉及数值操作的接口未校验正负
- (2) 越权操作：未验证当前用户是否有权执行某操作
- (3) 重放攻击：同一请求可重复提交导致多次扣款/充值
- (4) 条件竞争：多线程同时操作同一资源导致数据不一致
- (5) 顺序绕过：跳过某些步骤直接执行后续操作

1.3 本次实验环境

项目	说明
----	----

目标应用	基于Python Flask框架的用户信息管理平台
数据库	SQLite 3，使用data/users.db存储用户数据
已有功能	登录、注册、搜索、头像上传、个人中心、充值
本次新增功能	/profile（个人中心查看）、/recharge（余额充值）
漏洞类型	IDOR水平越权漏洞 + 业务逻辑漏洞（金额负值）
测试账号	admin/admin123（管理员，余额99999）、alice/alice2025（普通用户，余额100）

二、漏洞代码分析

2.1 漏洞1：IDOR水平越权漏洞

风险等级：高危 | **CWE-639**：通过用户控制的密钥绕过授权

2.1.1 漏洞位置

文件：app.py，路由 /profile

行号：约第900行

2.1.2 漏洞代码

```
@app.route("/profile")
@login_required
def profile():
    # 个人中心 - 存在IDOR漏洞
    # 【漏洞】user_id 从URL参数获取，用户可以任意修改
    user_id = request.args.get("user_id", "")

    # 【漏洞】直接根据传入的ID查询数据库，不验证身份
    user = _get_user_by_id(user_id)

    return render_template("profile.html", user=user)
```

2.1.3 漏洞成因分析

（1）身份标识来源错误：用户的身份标识（user_id）应从服务端的 session 中获取，而不是从客户端传来的 URL 参数中获取。session 中的数据由服务器签名保护，用户无法篡改；而 URL 参数用户可以随意修改。

（2）缺少权限校验：即使需要通过 user_id 查询，也应在查询前校验当前登录用户是否与目标 user_id 匹配。例如先查出当前用户自己的 user_id，再与目标 user_id 对比。

（3）未设置最小权限原则：系统在设计时没有遵循“用户只能访问自己的数据”这一基本原则。

2.1.4 攻击原理示意图

正常访问：

浏览器 → GET /profile?user_id=2 → 服务器查询ID=2 → 返回alice的资料 ✓

越权攻击：

浏览器 → GET /profile?user_id=1 → 服务器查询ID=1
→ 返回admin的资料！（越权成功！）✗

【原因】服务器没有检查"当前登录的用户是alice(ID=2)"
为什么能查到admin(ID=1)的资料

2.2 漏洞2：业务逻辑漏洞（金额负值）

风险等级：高危 | **CWE-841**：业务流程处理不当

2.2.1 漏洞位置

文件：app.py，路由 /recharge

行号：约第918行

2.2.2 漏洞代码

```
@app.route("/recharge", methods=["POST"])
@login_required
def recharge():
    # 充值 - 存在业务逻辑漏洞
    user_id = request.form.get("user_id", "")
    amount = request.form.get("amount", "0")

    # 【漏洞1】没有校验 amount 是否为正数
    # 攻击者可提交 amount=-100000 实现"反向扣款"

    # 【漏洞2】使用 f-string 拼接 SQL，存在SQL注入
    sql = f"UPDATE users SET balance = balance + {amount} WHERE id = {user_id}"
    c.execute(sql)
```

2.2.3 漏洞成因分析

（1）缺乏输入校验：开发者在设计充值功能时，默认地认为用户"只会"输入正数，而忘记了攻击者可以输入负数。这是典型的"信任用户输入"的安全误区。所有涉及金钱、积分、数量等数值型操作的接口，必须在服务端校验数值的范围和正负。

（2）SQL语句拼接：使用 f-string 直接将 amount 拼接到 SQL 语句中，既存在SQL注入风险，又使得 amount 的值可以在 SQL 层面被执行任意操作。amount=-100000 会变成 balance = balance + -100000，相当于 balance = balance - 100000。

（3）未使用参数化查询：参数化查询不仅防止SQL注入，也强制数据与代码分离，使数据库能够正确区分"数据"和"代码"。

三、攻击复现（POC）

3.1 POC 1：IDOR越权攻击

实验目标

以普通用户 alice 身份登录，通过修改URL参数查看管理员 admin 的个人信息。

实验步骤与结果

步骤	操作	请求	结果
1	以alice身份登录系统	POST /login username=alice&password=alice2025	登录成功，获取 session
2	查看自己的个人中心	GET /profile?user_id=2	正常显示 alice 的资料
3	修改URL参数查看 admin	GET /profile?user_id=1	⚠️ 成功看到admin的资料！ 邮箱、手机号、余额全部泄露
4	遍历其他用户ID	GET /profile?user_id=3 GET /profile?user_id=4	可查看所有已注册用户的资料

攻击结果

alice（普通用户，user_id=2）通过将URL参数从 user_id=2 修改为 user_id=1，成功获取了管理员 admin 的个人信息，包括邮箱（admin@example.com）、手机号（13800138000）和当前余额。本次攻击属于水平越权（同级别用户之间越权访问），若进一步结合其他漏洞，可升级为垂直越权（普通用户获取管理员权限）。

3.2 POC 2：业务逻辑漏洞攻击

实验目标

利用充值接口未校验金额正负的缺陷，通过提交负值实现非预期扣款。

实验步骤与结果

步骤	操作	请求	余额变化
----	----	----	------

1	查看admin当前余额	GET /profile?user_id=1	初始余额：99999
2	正常充值500元	POST /recharge user_id=1&amount=500	99999 + 500 = 100499
3	恶意提交负值-100000	POST /recharge user_id=1&amount=-100000	 100499 - 100000 = 499 ! 余额被"盗刷"扣除

攻击结果

攻击者通过提交负值 amount=-100000，成功将admin账户的余额从100499减少到499。这在本质上相当于"盗刷"了系统10万元。如果攻击者反复利用此漏洞，可以将任意用户的余额清零甚至变为负数。与此同时，amount参数还存在SQL注入风险，攻击者可在amount中注入恶意SQL代码获取数据库完全控制权。

四、修复方案

4.1 修复IDOR漏洞

核心修复原则：永远从服务端**session**获取用户身份，绝不信任客户端传入的用户标识。

修复后代码

```
@app.route("/profile")
@login_required
def profile():
    # 个人中心 - 已修复：从session获取用户
    # ✅ 从 session 获取当前登录的用户名
    # session中的数据由服务器签名，用户无法篡改
    username = session.get("username")

    # ✅ 根据用户名（不是user_id）查询用户信息
    c.execute("SELECT * FROM users WHERE username = ?", (username,))
    user = c.fetchone()

    return render_template("profile.html", user=user)
```

修复要点

要点	说明
1	删除从URL参数获取 user_id 的代码
2	从 session 中获取当前已登录的用户身份
3	用户只能查询自己的数据，无法通过修改参数越权
4	后续如需扩展（如管理员查看所有用户），应单独增加角色校验

4.2 修复业务逻辑漏洞

核心修复原则：所有涉及金钱的业务操作必须校验数值范围，且使用参数化查询。

修复后代码

```
@app.route("/recharge", methods=["POST"])
@login_required
def recharge():
    # 充值 - 已修复：校验金额+参数化查询
    # ✅ 从 session 获取当前用户
    username = session.get("username")
```

```
amount = int(request.form.get("amount", "0"))

#  修复1: 校验金额必须为正数
if amount <= 0:
    return "充值金额必须大于0"

#  修复2: 使用参数化查询防止SQL注入
c.execute(
    "UPDATE users SET balance = balance + ? WHERE username = ?",
    (amount, username)
)
```

五、修复前后代码对比

5.1 个人中心路由（/profile）对比

对比维度	修复前（存在漏洞）	修复后（安全）
用户来源	<code>request.args.get("user_id")</code> ← 从URL参数获取	<code>session.get("username")</code> ← 从服务器session获取
权限校验	无任何权限校验 直接根据传入ID查询	自动权限隔离 只能查自己的数据
查询方式	<code>f"WHERE id = {user_id}"</code> ← f-string拼接SQL	<code>WHERE username = ?</code> ← 参数化查询
攻击可能性	修改user_id即可越权 注入SQL代码	无法越权 无法注入SQL

5.2 充值路由（/recharge）对比

对比维度	修复前（存在漏洞）	修复后（安全）
金额校验	无校验 amount可直接为负	<code>if amount <= 0: return错误</code> 金额必须为正数
SQL语句	<code>f"balance + {amount}"</code> ← f-string拼接	<code>balance + ?</code> ← 参数化查询
用户身份	user_id从表单获取 可操作任意用户余额	username从session获取 只能操作自己的余额
攻击可能性	amount=-99999扣余额 amount=1;DROP TABLE删库	无法负值操作 无法注入SQL

六、修复后验证测试

6.1 IDOR越权测试

序号	测试场景	测试操作	预期结果	实际结果
1	alice查看自己的资料	GET /profile	显示alice的信息	[OK] 通过
2	alice尝试查看admin	GET /profile?user_id=1	仍显示alice的信息	[OK] 越权被拦截
3	alice尝试查看不存在的ID	GET /profile?user_id=999	仍显示alice的信息	[OK] 通过
4	admin查看自己的资料	GET /profile	显示admin的信息	[OK] 通过
5	未登录访问/profile	GET /profile (无Cookie)	重定向到登录页	[OK] 通过

6.2 业务逻辑测试

序号	测试场景	测试操作	预期结果	实际结果
1	正常充值100元	POST amount=100	余额增加100	[OK] 通过
2	负值充值	POST amount=-99999	报错：金额必须大于0	[OK] 被拦截
3	零值充值	POST amount=0	报错：金额必须大于0	[OK] 被拦截
4	超长字符串	POST amount=abc	报错：金额格式错误	[OK] 被拦截
5	SQL注入尝试	POST amount=1;DROP TABLE	参数化查询拦截	[OK] 安全

七、总结与安全开发指南

7.1 漏洞归纳

漏洞类型	OWASP Top 10	CWE编号	根因	攻击难度	危害等级
IDOR/水平越权	A01:2021 访问控制失效	CWE-639	信任前端传入的用户ID	极低（改URL即可）	高危
业务逻辑漏洞	A01:2021 访问控制失效	CWE-841	未校验金额正负	极低（改表单值即可）	高危

7.2 安全开发五原则

原则	名称	说明
1	永不信任用户输入	URL参数、表单数据、Cookie、HTTP头等所有客户端传入的数据均不可信，必须在服务端进行校验
2	服务端身份验证	用户身份应从 Session/Token 中获取，session数据由服务器签名保护，用户无法篡改
3	业务参数校验	金额、数量、积分等数值型业务数据，必须校验其范围（>0、<=最大值等）和格式
4	最小权限原则	用户只能操作属于自己的数据。每个请求在处理前，都应验证发起者是否有权执行该操作
5	参数化查询	所有数据库操作必须使用参数化查询（Prepared Statement），杜绝任何形式的字符串拼接

7.3 本次修复涉及的文件清单

文件路径	修改内容	行数变化
app.py	/profile路由重写（session替换URL参数）；/recharge增加金额校验和参数化查询	+30行
templates/profile.html	移除user_id隐藏字段	-1行
templates/base.html	导航栏链接从/profile?user_id=1改为/profile	0行

templates/ index.html	首页快捷入口链接修改	0行
--------------------------	------------	----

7.4 学习心得

通过本次实验，我深刻认识到：安全漏洞并不总是需要高深的技术才能利用。IDOR漏洞只需要修改URL中的一个数字，业务逻辑漏洞只需要在表单中填一个负数——这些操作任何普通用户都能做到。真正的安全防线在于开发者的安全意识：永远假设用户会做最坏的操作，永远在服务端做最严格的校验。

权限提升和业务逻辑漏洞的修复并不复杂，但它们暴露出的问题——"信任用户输入"和"缺乏权限校验"——却是Web安全中最常见也最危险的设计缺陷。在后续的开发中，我会将"权限校验"和"输入校验"作为每个接口的必选步骤，而不是可有可无的附加项。

— 报告完 —

报告日期: 2026-07-10 | 课程: 权限提升与业务逻辑漏洞修复 | 安全标准: OWASP Top 10 (2021) A01